Chapter 8. Object-Oriented Techniques

8.0 Introduction

Java is an object-oriented (OO) language in the tradition of Simula-67, SmallTalk, and C++. It borrows syntax from C++ and ideas from SmallTalk. The Java API has been designed and built on the OO model and following many OO design patterns. The idea of "design patterns" originated with Christopher Alexander and colleagues at UC Berkeley's Faculty of Architecture, as a way of teaching budding architects how to better design homes and offices.[1] In the 1980s and early 1990s, computer science departments published catalogs of design patterns in computing. The 1995 book *Design Patterns* (by Erich Gamma et al., Addison-Wesley, often called the "Gang of Four" book [GoF], after the four authors) catalogued some two dozen of the best. These design patterns, such as Proxy, Strategy, and Delegate, are used throughout modern development; an understanding of these patterns will help you better understand the use of the Java API and improve the design of your own classes.

Most of the GoF book examples were originally written in C++. For Java developers, the pieces I wrote for Oracle's *Java Magazine* include articles on half a dozen or so of these patterns. The book *Head First Design Patterns* by Eric Freeman and Elisabeth Robson (O'Reilly) describes all 23 GoF patterns, with examples in Java in a very entertaining and memorable fashion.

Advice, or Mantras

I could give you any number of short bits of advice on object-oriented programming. A few recurring themes arise when learning the basics of Java, and I suggest reviewing them when learning advanced Java.

Use the API

I can't say this often enough. A lot of the things you need to do have already been done by the good folks who develop the standard Java library (and third-party libraries). And this grows with every release. Learning the API well is a good foundation for avoiding

that deadly "reinventing the flat tire" syndrome—coming up with a second-rate equivalent of a first-rate product that was available to you the whole time. In fact, part of this book's mission is to prevent you from reinventing what's already there. One example of this is the Collections API in java.util, discussed in Chapter 7. The Collections API has a high degree of generality and regularity, so there is often no need to invent your own data structuring code.

Learn exceptions to the rule

There are a few exceptions to the rule of using the API: the `clone()` method in `java.lang.Object` should generally *not* be used. If you need to copy an object, just write a copy method, or a *copy constructor*. For `List`s, use the `ArrayList(List)` copy constructor. If you need to copy an array, use the static `Arrays.copyOf()` method. Joshua Bloch's arguments against the `clone()` method in the book *Effective Java* (Addison-Wesley) are persuasive and should be read by any dedicated Java programmer. While you're at it, read that whole book.

Another exception is the `finalize()` method in `java.lang.Object()`. Don't use it. It has been deprecated since Java 9 and ultimately marked for removal. It isn't guaranteed to be invoked; but because it might get invoked, it will cause your dead objects not to be garbage collected, resulting in a memory leak. If you need some kind of cleanup, take responsibility for defining a method and invoking it before you let any object of that class go out of reference. You might call such a method `cleanUp()`. For application-level cleanup, see the recipe for shutdown hooks on my website.

Generalize

There is a trade-off between generality (and the resulting reusability), which is emphasized here, and the convenience of application specificity. If you're writing one small part of a very large application designed according to OO design techniques, you'll have in mind a specific set of use cases. On the other hand, if you're writing toolkit-style code, you should write classes with few assumptions about how they'll be used. Making code easy to use from a variety of programs is the route to writing reusable code.

Read and write Javadoc

You've no doubt looked at the Java online documentation in a browser, in part because I just told you to learn the API well. Do you think Sun/Oracle hired millions of tech writers to produce all that documentation? No. That documentation exists because the developers of the API took the time to write Javadoc comments, those funny `/**` comments

you've seen in code. So, one more bit of advice: use Javadoc. The standard JDK provides a good standard mechanism for API documentation. And use it as you write the code—don't think you'll come back and write it in later. Tomorrow never arrives.

See Recipe 1.7 for details on using Javadoc.

Use subclassing and delegation

Use subclassing. But don't overuse subclassing. It is one of the best ways to not only avoid code duplication but also to develop software that works. See any number of good books on the topic of object-oriented design and programming for more details.

There are several alternatives to subclassing. One alternative is delegation. Think about "is a" versus "has a." For example, instead of subclassing `NameAndAddress` to make `BusinessPartner` and `Customer`, make `BusinessPartner` and `Customer` have instances of `NameAndAddress`. That is a clearer structure; having `BusinessPartner` *be a* `NameAndAddress` just because the partner *has a* name and address would not necessarily make sense. Delegation also makes it easier for a `Customer` to have both a billing address and a shipping address.

Another alternative is aspect-oriented programming (AOP), which allows you to bolt on extra functionality from the outside of your classes. AOP is provided by the Jakarta EE using EJB interception and by the Spring Framework AOP mechanism, both of which fall outside the scope of this book.

Use design patterns

In the introduction to this chapter, I mentioned design patterns as one of the important topics in object-oriented programming. Popularized by the Gang of Four (GoF) book and *Head First Design Patterns*, the idea provides a powerful catalog of things that programmers often reinvent. A design pattern provides a statement of a problem and its solution(s), rather like the present book, but generally at a higher level of abstraction. It is as important for giving a standard vocabulary of design as it is for its clear explanations of how the basic patterns work and how they can be implemented.

Table 8-1 shows some example uses of design patterns in the standard API.

Table 8-1. Design patterns in the JavaSE API

Pattern name	Meaning	Examples in Java API
Command	Encapsulate requests, allowing queues of requests, undoable operations, etc.	`javax.swing.Action`; `javax.swing.undo.UndoableEdit`
Decorator	One class decorates another	Swing `Border`s
Factory Method	One class makes up instances for you, controlled by subclasses	`getInstance` (in `Calendar`, `Format`, `Locale`...); `SocketFactory`; RMI `InitialContext`
Iterator	Loop over all elements in a collection, visiting each exactly once	`Iterator`; older `Enumeration`; `java.sql.ResultSet`
Model-View-Controller	Model represents data; View is what the user sees; Controller responds to user requests	`ActionListener` and friends; `Observer`/`Observable`; used internally by all visible Swing components
Proxy	One object stands in for another	RMI, AOP, Dynamic Proxy
Singleton	Only one instance may exist	`java.lang.Runtime`, `java.awt.Toolkit`

I have written articles on the State, Proxy, Command, Decorator, and Visitor patterns for Oracle's *Java Magazine*.

8.1 Object Methods: Formatting Objects with toString(), Comparing with Equals

Problem

You want your objects to have a useful default formating capability and to behave themselves when placed in `Collections` classes.

Solution

There are three important overridable methods inherited from `java.lang.Object`; of these, `toString()` provides default formatting, while `equals()` and `hashCode()` provide equality testing and efficient usage in `Map` implementations. Some others, such as `clone()` and `finalize()` and the `wait()` versions, are not recommended for general use.

Discussion

toString()

Whenever you pass an object to `System.out.println()` or any equivalent method, or involve it in string concatenation, Java automatically calls its `toString()` method. Java knows that every object has a `toString()` method because `java.lang.Object` has one, and all classes are ultimately subclasses of `Object`. The default implementation, in `java.lang.Object`, is neither pretty nor interesting: it just prints the class name, an @ sign, and the object's `hashCode()` value. For example, if you run the code:

```
public class ToStringWithout {
    int x, y;

    /** Simple constructor */
    public ToStringWithout(int anX, int aY) {
        x = anX; y = aY;
    }

    /** Main just creates and prints an object */
    public static void main(String[] args) {
        System.out.println(new ToStringWithout(42, 86));
    }
}
```

you might see this uninformative output:

```
ToStringWithout@990c747b
```

If you want to make objects of your class print better, provide an implementation of `toString()` that prints the class name and some of the important states; in fact, this is good practice for all but the most trivial classes. This gives you formatting control in `println()`, in debuggers, and anywhere your objects get referred to in a `String` context. Here is the previous program rewritten with a `toString()` method:

```
public class ToStringWith {
  int x, y;

  /** Simple constructor */
  public ToStringWith(int anX, int aY) {
    x = anX; y = aY;
```

```
  }

  @Override
  public String toString() {
    return "ToStringWith[" + x + "," + y + "]";
  }

  /** Main just creates and prints an object */
  public static void main(String[] args) {
    System.out.println(new ToStringWith(42, 86));
  }
}
```

This version produces the more useful output:

```
ToStringWith[42,86]
```

This example uses `String` concatenation, but you may also want to use `String.format()` or `StringBuilder`; see Chapter 3.

Also note that the Java `record` type provides a default `toString()` method automatically (see "Java record type 16").

hashCode() and equals()

To ensure your classes work correctly when any client code calls `equals()` or when these objects are stored in `Map` or other `Collection`s classes, outfit your class with `equals()` and `hashCode()` methods.

How do you determine equality? For arithmetic or Boolean operands, the answer is simple: you test with the equals operator (`==`). For object references, though, Java provides both `==` and the `equals()` method inherited from `java.lang.Object`. The `==` operator can be confusing because it simply compares two object references to see if they refer to the same object. This is not the same as comparing the values of the objects themselves.

The inherited `equals()` method is also not as useful as you might imagine. Some people seem to start their lives as Java developers thinking that the default `equals()` magically does some kind of detailed, field-by-field, or even binary comparison of objects. But it does *not* compare fields! It just does the simplest possible thing: it returns the value of

an `==` comparison on the two objects involved. So, for any *value classes* you write, you probably have to write an `equals` method.[2] Note that both the `equals` and `hashCode` methods are used by `Map`s or hashes (such as `HashMap`; see Recipe 7.9). So if you think somebody using your class might want to use instances as keys in a `Map`, or even compare your objects, you owe it to them (and to yourself!) to implement both `equals()` and `hashCode()` and to implement them properly.[3]

Most IDEs know how to generate usable `equals()` and `hashCode()` methods, but it's worth your while to understand what these are doing, for the occasional case where you need to tweak the generated code. Most have some kind of `Generate hashCode() and equals()` menu item that will only do both at the same time, not let you generate `equals()` without `hashCode()` or vice versa.

Here are the rules for a correct `equals()` method:

It is reflexive

`x.equals(x)` must be true.

It is symmetrical

`x.equals(y)` must be true if and only if `y.equals(x)` is also true.

It is transitive

If `x.equals(y)` is true and `y.equals(z)` is true, then `x.equals(z)` must also be true.

It is idempotent (repeatable)

Multiple calls on `x.equals(y)` return the same value (unless state values used in the comparison are changed, as by calling a set method).

It is cautious

`x.equals(null)` must return false rather than accidentally throwing a `NullPointerException`.

In addition, beware of one common mistake: the argument to `equals()` must be declared as `java.lang.Object`, not the class it is in; this is so that polymorphism will work correctly (some classes may not have an `equals()` method of their own). To prevent this mistake, the `@Override` annotation is usually added to the `equals()` override, as mentioned in Recipe 1.8.

Here is a class that endeavors to implement these rules:

```
public class EqualsDemo {
   private int int1;
   private SomeClass obj1;

   /** Constructor */
   public EqualsDemo(int i, SomeClass o) {
      int1 = i;
      if (o == null) {
         throw new IllegalArgumentException("Data Object may not be null");
      }
      obj1 = o;
   }

   /** Default Constructor */
   public EqualsDemo() {
      this(0, new SomeClass());
   }

   /** Demonstration "equals" method */
   @Override
   public boolean equals(Object o) {
      if (o == this)                    ❶
         return true;

      if (o == null)                    ❷
         return false;

      // Of the correct class?
      if (o.getClass() != EqualsDemo.class) ❸
         return false;

      EqualsDemo other = (EqualsDemo)o; // OK, cast to this class

      // compare field-by-field            ❹
      if (int1 != other.int1)         // compare primitives directly
         return false;
      if (!obj1.equals(other.obj1))    // compare objects using their equals
         return false;
      return true;
   }

   // ...
```

❶ Optimization: if same object, true by definition.

❷ If other object null, false by definition.

❸ Compare class descriptors using `!=`; see following paragraph.

❹ Optimization: compare primitives first. May or may not be worthwhile; may be better to order by those most likely to differ—depends on the data and the usage.

Another common mistake to avoid: note the use of class descriptor equality (i.e., `o.getClass() != EqualsDemo.class`) to ensure the correct class, rather than via `instanceof`, as is sometimes erroneously done. The reflexive requirement of the `equals()` method contract pretty much makes it impossible to compare a subclass with a superclass correctly, so we now use class equality (see Chapter 17 for details on the class descriptor).

Here is a basic JUnit test (see Recipe 2.9) for the `EqualsDemo` class:

```
/** Some JUnit test cases for EqualsDemo.
 * Writing a full set is left as "an exercise for the reader".
 */
class EqualsDemoTest {

 /** an object being tested */
 EqualsDemo d1;
 /** another object being tested */
 EqualsDemo d2;

 /** Method to be invoked before each test method */
 @BeforeEach
 void setUp() {
  d1 = new EqualsDemo();
  d2 = new EqualsDemo();
 }

 @Test
 void symmetry() {
  assertEquals(d1, d1);
 }

 @Test
 void symmetric() {
  assertTrue(d1.equals(d2) == d2.equals(d1));
 }

 @Test
```

```
  void caution() {
    assertNotEquals(null, d1);
  }
}
```

With all that testing, what could go wrong? Well, some things still need care. What if the object is a *subclass* of `EqualsDemo`? We should test that it returns false in this case.

What else could go wrong? Well, what if either `obj1` or `other.obj1` is null? You might have just earned a nice shiny new `NullPointerException`. So you also need to test for any possible null values. Good constructors can avoid these `NullPointerException`s, as I've tried to do in `EqualsDemo`, or else test for them explicitly.

Finally, you should never override `equals()` without also overriding `hashCode()`, and the same fields must take part in both computations.

hashCode()

The `hashCode()` method is supposed to return an `int` that should uniquely identify any set of values in objects of its class.

A properly written `hashCode()` method will follow these rules:

It is repeatable

`hashCode(x)` must return the same `int` when called repeatedly, unless set methods have been called.

It is consistent with equality

If `x.equals(y)`, then `x.hashCode()` must == `y.hashCode()`.

Distinct objects should produce distinct `hashCodes`

If `!x.equals(y)`, it is not required that `x.hashCode()` != `y.hashCode()`, but doing so may improve performance of hash tables (i.e., hashes may call `hashCode()` before `equals()`).

The default `hashCode()` on the standard JDK returns a machine address, which conforms to the first rule. Conformance to the second and third rules depends, in part, on your `equals()` method. Here is a program that prints the hashcodes of a small handful of objects:

```
public class PrintHashCodes {

 /** Some objects to hashCode() on */
 protected static Object[] data = {
  new PrintHashCodes(),
  new java.awt.Color(0x44, 0x88, 0xcc),
  new SomeClass()
 };

 public static void main(String[] args) {
  System.out.println("About to hashCode " + data.length + " objects.");
  for (int i=0; i<data.length; i++) {
   System.out.println(data[i].toString() + " --> " +
    data[i].hashCode());
  }
  System.out.println("All done.");
 }
}
```

What does it print?

```
> javac -d . oo/PrintHashCodes.java
> java oo.PrintHashCodes
About to hashCode 3 objects.
PrintHashCodes@982741a0 --> -1742257760
java.awt.Color[r=68,g=136,b=204] --> -12285748
SomeClass@860b41ad --> -2046082643
All done.
>
```

The hashcode value for the `Color` object is interesting. It is actually computed as something like this:

```
alpha<<24 + r<<16 + g<<8 + b
```

In this formula, `r`, `g`, and `b` are the red, green, and blue components, respectively, and `alpha` is the transparency. Each of these quantities is stored in 8 bits of a 32-bit integer. If the alpha value is greater than 128, the high bit in this word—having been set by shifting into the sign bit of the word—causes the integer value to appear negative when printed as a signed integer. Hashcode values are of type `int`, so they are allowed to be negative.

Again, the `record` type provides `equals()` and `hashCode()`. These make all fields take part, so you might want to override them if that's not appropriate.

Difficulties and Alternatives to Clone

The `java.util.Observable` class (designed to implement the Model-View-Controller pattern with AWT or Swing applications) contains a private `Vector` but no clone method to deep-clone it. Thus, `Observable` objects cannot safely be cloned, ever!

This and several other issues around `clone()`—such as the uncertainty of whether a given `clone()` implementation is deep or shallow—suggest that `clone()` was not as well thought out as it might be. An alternative is simply to provide a copy constructor or similar method:

```
public class CopyConstructorDemo {
  public static void main(String[] args) {
    CopyConstructorDemo object1 = new CopyConstructorDemo(123, "Hello");
    CopyConstructorDemo object2 = new CopyConstructorDemo(object1);
    if (!object1.equals(object2)) {
      System.out.println("Something is terribly wrong...");
    }
    System.out.println("All done.");
  }

  private int number;
  private String name;

  /** Default constructor */
  public CopyConstructorDemo()  {
  }

  /** Normal constructor */
  public CopyConstructorDemo(int number, String name)  {
    this.number = number;
    this.name = name;
  }

  /** Copy Constructor */
  public CopyConstructorDemo(CopyConstructorDemo other)  {
    this.number = other.number;
    this.name = other.name;
  }
  // hashCode() and equals() not shown
```

8.2 Constructor Simplification: Statements Before super(...) 22P

Problem

In writing a constructor, you need to compute or test some values before calling the superclass constructor.

Solution

Use a Java 22 preview feature that allows nonfield computations before the call to `super()`.

Discussion

For most of its history, Java would not allow any code in a constructor before the call to `super()`. This is necessary for object construction to work properly as a chained process, to ensure that superclass fields are initialized before the subclass constructor can access them. However, the restriction was perhaps too strict. It is quite possible to do some operations that cannot affect the outcome of the superclass constructor. In Java 22 preview, JEP 447 "Statements before super()" allows these. Example 8-1 shows code that will only work on Java 22.

Example 8-1. main/src/main/java//oo/StatementsBeforeSuper.java

```
class One {
  int dom;
  One(int dom) {
    if (dom <= 0 || dom > 31)
      throw new IllegalArgumentException(
        "Day of Month out of range in call to One()");
    this.dom = dom;
  }
}

class Two extends One {
  Two() {
    var x = LocalDate.now().getDayOfMonth();
    super(x);
  }
```

```
}

void main() {
  var o = new Two();
  System.out.println("Day of Month is " + o.dom);
}
```

Running this example on a Java 22 JDK results in the following:

```
$ javac --enable-preview --source 22 oo/StatementsBeforeSuper.java
$ java --enable-preview -cp oo StatementsBeforeSuper
Day of Month is 5
$ date
Sun November  5 14:51:29 EDT 2023
$
```

Code before `super()` is considered to be in a "pre-construction context," and cannot access instance methods. The complexities thereof are explored in JEP 447, which introduced this enhancement into Java 22.

8.3 Using Inner Classes

Problem

You need to write a private class, or a class to be used in one other class at most.

Solution

Use a nonpublic class or an inner class.

Discussion

A nonpublic class can be written as part of another class's source file, but not inside that class. An inner class is Java terminology for a class defined inside another class. Inner classes were first popularized with early Java for use as event handlers for GUI applications, but they have a much wider application.

Inner classes can, in fact, be constructed in several contexts. An inner class defined as a member of a class can be instantiated anywhere in that class. An inner class defined in-

side a method can be referred to later only in the same method. Inner classes can also be named or anonymous. A named inner class has a full name that is compiler dependent; the standard JVM uses a name like `MainClass$InnerClass` for the resulting file. An anonymous inner class, similarly, has a compiler-dependent name; the JVM uses `MainClass$1`, `MainClass$2`, and so on.

These classes cannot be instantiated in any other context; any explicit attempt to refer to, say, `OtherMainClass$InnerClass`, is caught at compile time:

main/src/main/java/oo/AllClasses.java

```
public class AllClasses {
 public class Data {                    ❶
  int x;
  int y;
 }
 public void getResults() {
  JButton b = new JButton("Press me");
  b.addActionListener(new ActionListener() { ❷
   public void actionPerformed(ActionEvent evt) {
    Data loc = new Data();
    loc.x = ((Component)evt.getSource()).getX();
    loc.x = ((Component)evt.getSource()).getY();
    System.out.println("Thanks for pressing me");
   }
  });
 }
}

/** Class contained in same file as AllClasses, but can be used
 * (with a warning) in other contexts.
 */
class AnotherClass {                    ❸
 // methods and fields here...
 AnotherClass() {
  // Inner class from above cannot be used here, of course
  // Data d = new Data();  // EXPECT COMPILE ERROR
 }
}
```

❶ This is an inner class, which can be used anywhere in class `AllClasses`.

❷

This shows the anonymous inner class syntax, which uses `new` with a type followed by `(){`, a class body, and `}`. The compiler will assign a name; the class will extend or implement the given type, as appropriate.

❸ This is a nonpublic class; it can be used in the main class and (with warning) in other classes.

One issue is that the inner class retains a reference to the outer class. This is important if, for example, the inner class is an iterator that will be in use for a long time. If you want to avoid memory leaks if the inner class will be held for a longer time than the outer, you can make the inner class `static`. In that case it cannot, of course, access instance (nonstatic) methods or fields in the outer class.

TIP

Inner classes implementing a single-method interface—including the `ActionListener` in the preceding example—can be written in a much more concise fashion as lambda expressions (see Chapter 9).

8.4 Simplifying Data Objects with Records (or Lombok)

Problem

You spend time writing data classes that are Plain Old Java Objects (POJO), with boilerplate code such as setters and getters, `equals()`, and `toString()`.

Solution

In Java 16+, use the `record` data type, which generates 100% of the boilerplate methods for you. On older versions of Java, use Lombok to autogenerate the boilerplate methods.

Discussion

When Java was new, before there were good IDEs, developers had to write getters and setters by hand, or by copy-paste-change. Back then I did a study of one existing large codebase and found about a one-half of 1% failure rate. The setter stored the value in

the wrong place or the getter retrieved the wrong value. Assuming random distribution, this meant that one getter call in a hundred gave the wrong answer! The application still worked, so I must assume those wrong answers didn't matter; presumably, the broken methods weren't actually called.

Now we have IDEs that can generate all the boilerplate methods such as setters/getters, equals, `toString()`, and so on. But you still have to remember to invoke these generators.

Java record type 16

The `record` type introduced in Java 16 provides a good solution for classes whose objects rarely or never need to be modifed. A `record` is a construct for data classes, a restricted form of class. You need only write the name of a data object and its fields, and the compiler will provide a constructor, getters, `hashCode()` and `equals()`, and `toString()`:

```
public record Person(String name, String emailAddress) { }
```

The provided constructor has the same signature as the record declaration. All fields are implicitly final, and the `record` provides getters but not setters. Because there can *never* be setters, the getters have just the name of the field; they do not follow the JavaBeans pattern of capitalizing the field name and prepending `get`, e.g., `getName()`. Immutable objects are important for reliable code (see Chapter 9). You can provide other members such as extra constructors, static fields, and static or instance methods. Records are final, cannot be abstract, and cannot declare additional instance fields. This is all in keeping with the fact that the state of the object is as declared in the `record` header. Here I create a `Person` record and make an instance of it, all in JShell:

```
$ jshell
|  Welcome to JShell -- Version 16
|  For an introduction type: /help intro

jshell> record Person(String name, String email) {}

jshell> var p = new Person("Covington Roderick Smythe", "roddy@smythe.tld")
p ==> Person[name=Covington Roderick Smythe, email=roddy@smythe.tld]

jshell> p.name()
$3 ==> "Covington Roderick Smythe"
```

```
jshell>
```

One-line record definitions typically don't need to be in a source file all their own. To show a complete example, I baked the `Person` record into a new demo program `PersonRecordDemo`, shown in Example 8-2.

Example 8-2. main/src/main/java/record/PersonRecordDemo.java

```
public class PersonRecordDemo {

  /** This is all it takes to define a record -
   * compiler generates constructor, toString, equals/hashcode, etc.
   * Hint: after compiling this file, do:
   * javap 'PersonRecordDemo$Person'
   */
  public record Person(String name, String email) { }

  /**
   * Now we can use the record type as a regular data class.
   */
  public static void main(String[] args) {
    Person p = new Person("Covington Roderick Smythe", "roddy@smythe.tld");
    System.out.println(p);
  }
}
```

We can compile this file with `javac`, and then use `javap` to view the class's structure:

```
$ javac PersonRecordDemo.java
Note: PersonRecordDemo.java uses preview language features.
Note: Recompile with -Xlint:preview for details.
$ javap PersonRecordDemo'$'Person
Compiled from "PersonRecordDemo.java"
public final class PersonRecordDemo$Person extends java.lang.Record {
  public PersonRecordDemo$Person(java.lang.String, java.lang.String);
  public java.lang.String toString();
  public final int hashCode();
  public final boolean equals(java.lang.Object);
  public java.lang.String name();
  public java.lang.String email();
}
```

The *$* in the filename has to be escaped from the Unix shell. We see that the compiler has generated the constructor, `toString()`, `hashCode()` and `equals()`, and read-only accessors `name()` and `email()`.

Lombok

Project Lombok was an early solution to this problem, still used by many. Lombok processes classes containing for its annotations and rewrites the classes to have the chosen methods.

If you want to use Lombok, you will need to add the dependency `org.projectlombok:lombok:1.18.4` (or newer) to your build script. Or, if you are using an IDE, download the Lombok JAR file from the Project Lombok website and install it as per the instructions there. Then you can annotate your class with annotations like these:

```
@Setters @Getters
```

Presto! No more forgetting to generate these methods; Lombok will do the work for you.

Other annotations include the following:

```
@ToString
@EqualsAndHashCode
@AllArgsConstructor
```

For data classes, there is even `@Data`, which is a shortcut for `@ToString`, `@EqualsAndHashCode`, `@Getter` on all fields, `@Setter` on all nonfinal fields, and `@RequiredArgsConstructor`!

See Also

The original description of and rationale for the `record` mechanism is in JEP 359 at the OpenJDK website.

Lombok is implemented internally as an annotation processor, which runs at compile time. If you want more information on how this works, see Jacek Dubikowski's tutorial on annotation processors.

8.5 Providing Callbacks via Interfaces

Problem

You want to provide callbacks—that is, have unrelated classes call back into your code. For example, the observer class in the like-named pattern can be implemented by any class that implements the `Observer` interface.

Solution

One way to provide callbacks is to use a Java interface.

Discussion

An interface is a class-like entity that originally could contain only `abstract` methods and `final` fields. Later versions of Java had added `default` and `private` methods in interfaces. As we've seen, interfaces are used a lot in Java! In the standard API, the following are a few of the commonly used interfaces:

- `Runnable`, `Comparable`, and `Cloneable` (in `java.lang`).
- `List`, `Set`, `Map`, and `Enumeration/Iterator` (in the Collections API; as you'll see in Chapter 7).
- `ActionListener`, `WindowListener`, and others in the GUI layer.
- `Driver`, `Connection`, `Statement`, and `ResultSet` in JDBC (not officially an acronym, but widely understood as Java Database Connectivity); see the information on my website.
- The *remote interface*—the contract between the client and the server—is specified as an `Interface` (in some less-common Java Enterprise API network schemes: RMI [Remote Method Invocation], CORBA [Common Object Request Broker Interface], and EJB [Enterprise JavaBean]).

SUBCLASS, ABSTRACT CLASS, OR INTERFACE?

There is usually more than one way to solve a problem. Some problems can be solved by subclassing, by use of abstract classes, or by interfaces. The following general guidelines may help:

- A class can only extend one other class, but it can implement any number of interfaces; keep this in mind when deciding to use abstract classes or interfaces.
- *Use an abstract class* when you want to provide a template for a series of subclasses, all of which may inherit some of their functionality from the parent class but are required to implement some of it themselves. (Any subclass of a geometric `Shape` class might have to provide a `computeArea()` method; because the top-level `Shape` class cannot do this, it would be abstract. This is implemented in Recipe 8.6.)
- *Use an interface with default methods* for many of the same things you'd have used an abstract class for prior to Java 8, which introduced default methods. Default methods can be added to an interface without breaking existing code, such as `forEach` and `of` methods being added to the `Collection` interface so that every `List` instance has a `forEach` method built in. One limitation is that interfaces cannot have nonpublic nondefault methods, as these are made public by default in interfaces.
- *Subclass* when you need to extend a class and add some functionality to it, whether the parent class is abstract or not. See the standard Java APIs and the examples in Recipes 2.9, 8.6, and 10.11
- *Subclass* when you are required to extend a given class. Some APIs such as *Servlets* use subclassing to ensure base functionality in classes that are dynamically loaded (see Recipe 17.1).
- *Define an interface* when there is no common parent class with the desired functionality and when you want only certain unrelated classes to have that functionality (see the `PowerSwitchable` interface later in this recipe). You should also choose this option if you know that you'll need (or think there is a chance you might later need) to be able to pass in unrelated classes for testing purposes. Using mock objects is a very common strategy in unit testing. Some say that interfaces should be your first choice at least as often as subclassing.
- *Use interfaces as markers* to indicate something about a class. Marker interfaces commonly have no abstract methods. The standard API, for example, uses `Serializable` as a marker interface to indicate permission to serialize objects of the implementing class. See Recipe 14.6 for information on serialization.

Suppose we are generating a building management system. To be energy efficient, we want to be able to remotely turn off (at night and on weekends) such things as room

lights and computer monitors, which use a lot of energy. Assume we have some kind of remote control technology. It could be a commercial version of BSR's house-light control technology X10, it could be Bluetooth or 802.11—it doesn't matter. What matters is that we have to be very careful what we turn off. It would cause great ire if we turned off computer processors automatically—people often leave things running overnight. It would be a matter of public safety if we ever turned off the building's emergency lighting.[4]

So we've come up with the design shown in Figure 8-1.

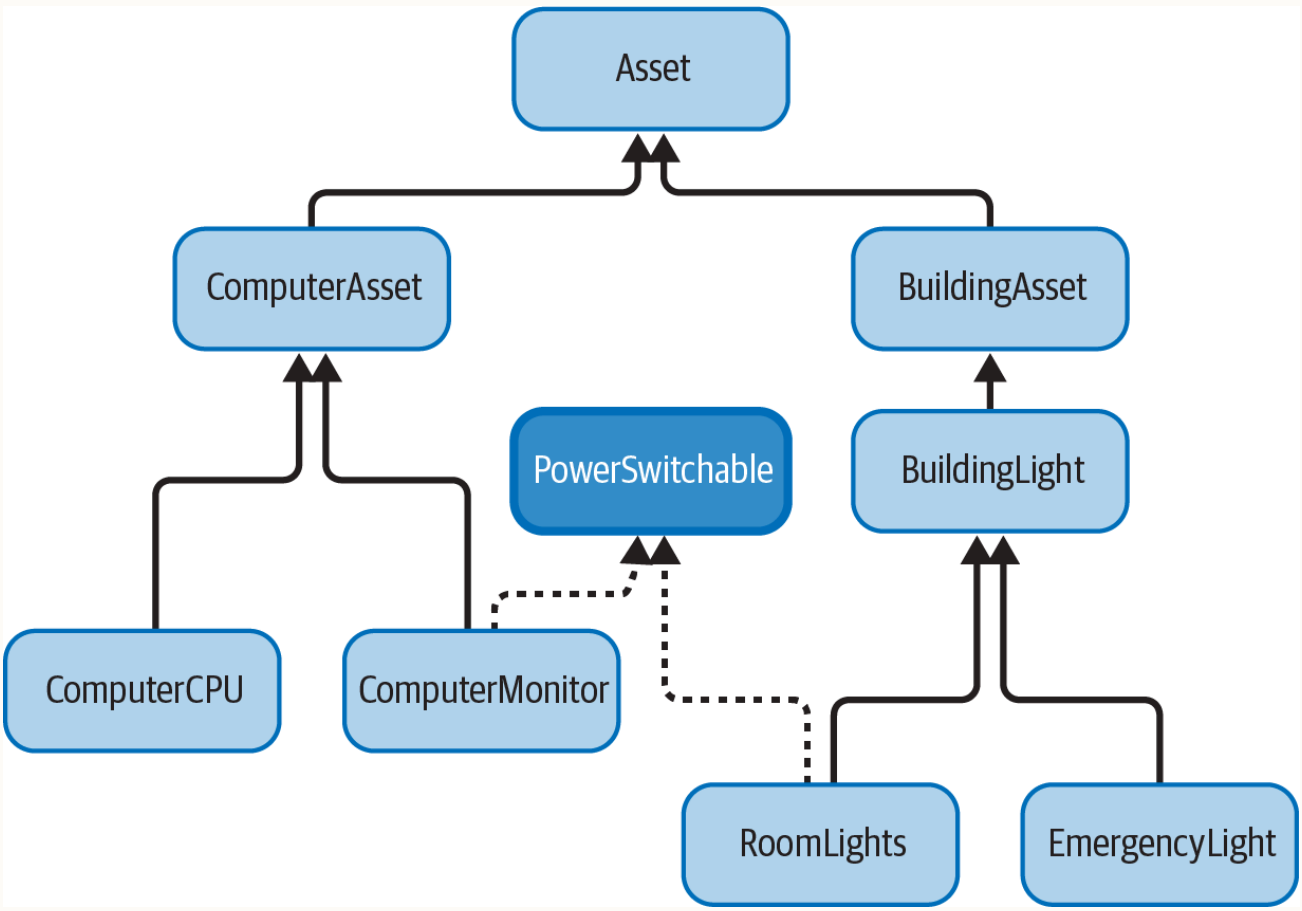


Figure 8-1. Classes for a building management system

The code for these data classes is not shown (it's pretty trivial), but it's in the *oo/interfaces* directory of the online source. The top-level classes (i.e., `BuildingLight` and `Asset`) are abstract classes. You can't instantiate them because they don't have any specific functionality. To ensure—both at compile time and at runtime—that we can never switch off the emergency lighting, we need only ensure that the class representing it, `EmergencyLight`, does not implement the `PowerSwitchable` interface.

Note that we can't very well use direct inheritance here. No common ancestor class includes both `ComputerMonitor` and `RoomLights` that doesn't also include `ComputerCPU` and `EmergencyLight`. Use interfaces to define functionality in unrelated classes.

How we use these is demonstrated by the `BuildingManagement` class; this class is not part of the hierarchy shown in Figure 8-1, but it *uses* a collection of `Asset` objects from that hierarchy.

Items that can't be switched off must nonetheless be in the database for various purposes (auditing, insurance, etc.). In the method that turns things off, the code is careful to check whether each object in the database is an instance of the `PowerSwitchable` interface. If so, the object is casted to `PowerSwitchable` so that its `powerDown()` method can be called. If not, the object is skipped, thus preventing any possibility of turning out the emergency lights or shutting off a machine that is busy running SETI@home, downloading a big MP3 playlist, or performing system backups. The following code shows this set of classes in action:

```
public class BuildingManagement {

 List<Asset> things = new ArrayList<>();

 /** Scenario: goodNight() is called from a timer Thread at 2200, or when
  * we get the "shutdown" command from the security guard.
  */
 public void goodNight() {
  things.forEach(obj -> {
    if (obj instanceof PowerSwitchable switchable) // Java 14 way
      switchable.powerDown();
    });
 }

 public void goodNightFunctional() {
  things.stream().filter(obj -> obj instanceof PowerSwitchable)
    .forEach(obj -> ((PowerSwitchable)obj).powerDown());
 }

 // goodMorning() would be similar, but call each one's powerUp().

 /** Add a Asset to this building */
 public void add(Asset thing) {
  System.out.println("Adding " + thing);
  things.add(thing);
 }

 /** The main program */
 public static void main(String[] av) {
  BuildingManagement b1 = new BuildingManagement();
  b1.add(new RoomLights(101));  // control lights in room 101
```

```
        b1.add(new EmergencyLight(101));  // and emerg. lights.
        // add the computer on desk#4 in room 101
        b1.add(new ComputerCPU(10104));
        // and its monitor
        b1.add(new ComputerMonitor(10104));

        // time passes, and the sun sets...
        b1.goodNight();
    }
}
```

When you run this program, it shows all the items being added but only the `PowerSwitchable` ones being switched off:

```
> java oo.interfaces.BuildingManagement
Adding RoomLights@2dc77f32
Adding EmergencyLight@2e3b7f32
Adding ComputerCPU@2e637f32
Adding ComputerMonitor@2f1f7f32
Dousing lights in room 101
Dousing monitor at desk 10104
>
```

8.6 Polymorphism/Abstract Methods

Problem

You want each of a number of subclasses to provide its own version of one or more methods.

Solution

Make the method abstract in the parent class; this makes the compiler ensure that it is implemented in each subclass.

Discussion

A hypothetical drawing program uses a `Shape` subclass for anything that is drawn. `Shape` has an abstract method called `computeArea()` that computes the exact area of the given shape:

```
public abstract class Shape {
   protected int x, y;
   public abstract double computeArea( );
}
```

A `Rectangle` subclass, for example, has a `computeArea()` that multiplies width times height and returns the result:

```
public class Rectangle extends Shape {
   double width, height;
   public double computeArea( ) {
      return width * height;
   }
}
```

A `Circle` subclass returns πr^2:

```
public class Circle extends Shape {
   double radius;
   public double computeArea( ) {
      return Math.PI * radius * radius;
   }
}
```

This system has a high degree of generality. In the main program, we can iterate over a collection of `Shape` objects and—here's the real beauty—call `computeArea()` on any `Shape` subclass object without having to worry about what kind of shape it is. Java's polymorphic methods automatically call the correct `computeArea()` method in the class of which the object was originally constructed:

main/src/main/java/oo/shapes/ShapeDriver.java

```
/** Part of a main program using Shape objects */
public class ShapeDriver {

 Collection<Shape> allShapes;  // created in a Constructor, not shown

 /** Iterate over all the Shapes, getting their areas;
  * this cannot use the Java 8 Collection.forEach because the
  * variable total would have to be final, which would defeat the purpose :-)
```

```
     */
    public double totalAreas() {
      double total = 0.0;
      for (Shape s : allShapes) {
        total += s.computeArea();
      }
      return total;
    }
```

Polymorphism is a great boon for software maintenance: if a new subclass is added, the code in the main program does not change. Further, all the code that is specific to, say, polygon handling, is all in one place: in the source file for the `Polygon` class. This is a big improvement over older languages, where type fields in a structure were used with case or `switch` statements scattered all across the software. Java makes software more reliable and maintainable with the use of polymorphism.

8.7 Improving Interfaces with Default, Static, and Private Methods

Problem

You want to use the recently added features in interface types.

Solution

Use default, static, and/or private methods.

Discussion

When Java was young, interfaces could only have abstract methods. Over the years they have gained several other abilities.

Default methods

Default methods were added in Java 8, and they allow for the addition of new methods in existing interfaces without breaking all implementations. This has been used in several interfaces, including some popular ones in the Collections Framework (see Recipe 7.4). For example, as of Java 8, the `Iterable` interface has a `forEach` method that was not present early on; because this is a `default` method, all existing `List` and other

`Collection` implementations gained the `forEach` method, without needing any changes! This is a prime example of how the Java core team keeps evolving the language without breaking backward compatibility. A simple implementation of this inside the JDK source `Iterable.java` would have been:

```
public interface Iterable {
      // ...
      default void forEach(Consumer c) {
            for (int i = 0; i < length(); i++) {
                  c.accept(get(i));
            }
      }
}
```

Since `Iterable`s are in fact iterable, the actual code in the JDK is even simpler—the loop is simply:

```
for (T t : this) {
      c.accept(t);
}
```

Also note that, since all interface methods are now public, it is considered redundant to put the `public` modifier on any interface method, although omitting it is not yet universal.

Static and private methods in interfaces

Java 8 also allows interfaces to have static methods. These also must have a body, must be called by the interface's name, and are also implicitly public.

In Java 9, interfaces further gained the ability to have private methods. These are meant to be used from within `default` methods, and of course cannot be called from outside the interface.

Example 8-3 shows some examples of these changes.

Example 8-3. main/src/main/java/structure/InterfaceAdditions.java

```
public class InterfaceAdditions implements Demo {
  void main() { // Instance main in a named class
```

```
    // Invoke public interface method in implementing class
    System.out.println(getStatus());
    // Call static interface method
    System.out.println(Demo.getPresent());
    // Following cannot compile, private method is
    // only visible in the interface, not impl classes.
    // var s = innerStatus();
  }
}

interface Demo {

  default String getStatus() { return innerStatus(); }

  static String getPresent() { return "Freebie"; }

  private String innerStatus() { return "OK"; }
}

class InterfaceChangesII implements Demo {
  // Forllowing is a valid override, but must be public
  public String getStatus() { return "Unknown"; }
}
```

Languages that support multiple inheritance of classes, such as C++, have to deal with the issue of conflicting method names inherited from different classes, the so-called diamond inheritance problem, named as such because the inheritance tree forms a diamond shape. If there are conflicting methods in Java interfaces, the compiler will detect them, and you'd use `interfacename.super.methodName()` to disambiguate. There's an example in the online code in *main/src/main/java/lang/InterfaceConflicts.java*.

8.8 Using Typesafe Enumerations

Problem

You need to manage a small list of discrete values within a program.

Solution

Use the Java `enum` mechanism.

Discussion

To enumerate means to list all the values. You often know that a small list of possible values is all that's wanted in a variable, such as the months of the year, the suits or ranks in a deck of cards, or the primary and secondary colors. The C programming language provided an `enum` keyword:

```
enum { BLACK, RED, ORANGE} color;
```

Java was criticized in its early years for its lack of enumerations, which many developers have wished for. Many have had to develop custom classes to implement the *typesafe enumeration pattern*.

But C enumerations are not typesafe; they simply define constants that can be used in any integer context. For example, this code compiles without warning, even on older versions of GCC (those before 6.1.0) with `-Wall` (all warnings), whereas a C++ compiler catches the error:[5]

```
enum { BLACK, RED, ORANGE} color;
enum { READ, UNREAD } state;

/*ARGSUSED*/
int main(int argc, char *argv[]) {
        color = RED;
        color = READ; // In C this will compile, give bad results
        return 0;
}
```

To replicate this mistake in Java, one needs only to define a series of `final int` values; it will still not be *typesafe*. By typesafe I mean that you cannot accidentally use values other than those defined for the given enumeration. The definitive statement on the typesafe enumeration pattern is probably the version defined in item 21 of Joshua Bloch's book *Effective Java* (Addison-Wesley). All modern Java versions include enumerations; it is no longer necessary to use the code from Bloch's book. Bloch was one of the authors of the typesafe enumeration specification (`enum` keyword), so you can be sure that Java now does a good job of implementing his pattern. These `enum`s are implemented as classes, subclassed (transparently, by the compiler) from the class `java.lang.Enum`. Unlike C, and unlike a series of `final int`s, Java typesafe enumerations have the following qualities:

- They are printable (they print as the name, not as an underlying `int` implementation).
- They are almost as fast as `int` constants, but the code is more readable.
- They can be easily iterated over.
- They use a separate namespace for each `enum` type, which means you don't have to prefix each with some sort of constant name, like `ACCOUNT_SAVINGS`, `ACCOUNT_CHECKING`, etc.

`Enum` constants are not compiled into clients, giving you the freedom to reorder the constants within your `enum` without recompiling the client classes. That does not mean you should do this, however; think about the case where objects that use the enums have been persisted, and the person designing the database mapping used the numeric values of the `enum`s. Bad idea to reorder then!

Additionally, an `enum` type is a class, so it can, for example, implement arbitrary interfaces; and you can add constructors, fields, and methods to an `enum` class.

Compared to Bloch's Typesafe Enum pattern in the book:

- Java `enum`s are simpler to use and more readable (those in the book require a lot of methods, making them cumbersome to write).
- `Enum`s can be used in `switch` statements.

So there are many benefits and few pitfalls.

The `enum` keyword is at the same level as the keyword `class` in declarations. That is, an `enum` may be declared in its own file with public or default access. It may also be declared inside classes, much like nested or inner classes (see Recipe 8.3). *Media.java*, shown in Example 8-4, is a code sample showing the definition of a typesafe `enum`.

Example 8-4. structure/Media.java

```
public enum Media {
   BOOK, MUSIC_CD, MUSIC_VINYL, MOVIE_VHS, MOVIE_DVD;
}
```

Notice that an `enum` class *is* a class; see what `javap` thinks of the `Media` class:

```
C:&gt; javap Media
Compiled from "Media.java"
```

```
public class Media extends java.lang.Enum{
  public static final Media BOOK;
  public static final Media MUSIC_CD;
  public static final Media MUSIC_VINYL;
  public static final Media MOVIE_VHS;
  public static final Media MOVIE_DVD;
  public static final Media[] values( );
  public static Media valueOf(java.lang.String);
  public Media(java.lang.String, int);
  public int compareTo(java.lang.Enum);
  public int compareTo(java.lang.Object);
  static {};
}
```

Product.java, shown in Example 8-5, is a code sample that uses the `Media` enum.

Example 8-5. main/src/main/java/structure/Product.java

```
public class Product {
 String title;
 String artist;
 Media  media;

 public Product(String artist, String title, Media media) {
  this.title = title;
  this.artist = artist;
  this.media = media;
 }

 @Override
 public String toString() {
  switch (media) {
  case BOOK:
   return title + " is a book";
  case MUSIC_CD:
   return title + " is a CD";
  case MUSIC_VINYL:
   return title + " is a relic of the age of vinyl";
  case MOVIE_VHS:
   return title + " is on old video tape";
  case MOVIE_DVD:
   return title + " is on DVD";
  default:
   return title + ": Unknown media " + media;
  }
```

```
  }
}
```

In Example 8-6, `MediaFancy` shows how operations (methods) can be added to enumerations; the `toString()` method is overridden for the `Book` value of this enum.

Example 8-6. main/src/main/java/structure/MediaFancy.java

```
/** An example of an enum with method overriding */
public enum MediaFancy {
  /** The enum constant for a book, with a method override */
  BOOK {
    public String toString() { return "Book"; }
  },
  /** The enum constant for a Music CD */
  MUSIC_CD,
  /** ... */
  MUSIC_VINYL,
  MOVIE_VHS,
  MOVIE_DVD;

  /** It is generally discouraged to have a main() in an enum;
   * please forgive this tiny demo class for doing so.
   */
  public static void main(String[] args) {
    MediaFancy[] data =  { BOOK, MOVIE_DVD, MUSIC_VINYL };
    for (MediaFancy mf : data) {
      System.out.println(mf);
    }
  }
}
```

Running the `MediaFancy` program produces this output:

```
Book
MOVIE_DVD
MUSIC_VINYL
```

That is, the `Book` values print in a user-friendly way compared to the default way the other values print. In real life you'd want to extend this to all the values in the `enum`.

Finally, `EnumList`, in Example 8-7, shows how to list all the possible values that a given `enum` can take on; simply iterate over the array returned by the enumeration class's inherited `values()` method.

Example 8-7. structure/EnumList.java

```
public class EnumList {
  enum State {
    ON, OFF, UNKNOWN
  }
  public static void main(String[] args) {
    for (State i : State.values()) {
      System.out.println(i);
    }
  }
}
```

The output of the `EnumList` program is this, of course:

```
ON
OFF
UNKNOWN
```

8.9 Using Type Pattern Matching

Problem

You want to simplify code that processes multiple types of data.

Solution

Use type pattern matching.

Discussion

The concept of *type pattern matching* is usually referred to as just *pattern matching*. Unfortunately, at a glance this could be confused with regular expression pattern matching, which is covered throughout Chapter 4. Type pattern matching applies to classes

and class-like things, and starting with Java 21 (previously in preview) it can be used in `switch` statements to greatly simplify code.

Imagine a simple calculator program, in which the parser has constructed objects whose types represent operations such as `Add`, `Subtract`, and so on, all of which extend class `Op`. A second pass will evaluate and print each operation. Prior to type pattern matching in `switch` (introduced in Java 21) the code might look like Example 8-8.

Example 8-8. main/src/main/java/lang/PatternMatchingString.java (old way)

```
System.out.println("Prior to type pattern matching:");
for (Op op : ops) {
  if (op instanceof Add) {
    System.out.println(((Add)op).left + ((Add)op).right);
  }
  else if (op instanceof Subtract) {
    System.out.println(((Subtract)op).left - ((Subtract)op).right);
  }
  else if (op instanceof Multiply) {
    System.out.println(((Multiply)op).left * ((Multiply)op).right);
  }
  else if (op instanceof Divide) {
    System.out.println(((Divide)op).left / ((Divide)op).right);
  }
  else System.out.println("Dunno about " + op);
}
```

This works, but it is fairly verbose. With Java 21, the code could be written more concisely, as in Example 8-9.

Example 8-9. main/src/main/java/lang/PatternMatchingString.java (new way)

```
System.out.println("Using type pattern matching:");
for (Op op : ops) {
  System.out.println(
    switch(op) {
    case null      -> "Null is not a valid Op!";
    case Add      a -> a.left + a.right;
    case Subtract s -> s.left - s.right;
    case Multiply m -> m.left * m.right;
    case Divide   d -> d.left / d.right;
    }
```

```
    );
  }
```

For example, the code `case Add a -> a.left + a.right` says that if `op` is of type `Add`, then implicitly cast it to type `Add`, to be able to refer to its fields without an explicit cast.

19 Pattern matching can also be used to deconstruct or destructure `record`s. Here again we put the variables after the type, but in this form the variables can be a `record` type with a list of fields to extract. This is shown, both using `instanceof` and using a `switch`, in Example 8-10.

Example 8-10. main/src/main/java/lang/PatternRecordDestruction.java

```
record Graduate(String name, String degree, int year){}

void main() {

  // with instanceof:
  for (Object o : grads) {
    if (o instanceof Graduate(var nm, var deg, var year)) {
      System.out.printf("%s graduated %d with a %s\n", nm, year, deg);
    }
  }

  // pattern match in switch:
  for (Object o : grads) {
    switch(o) {
      case Graduate(var nm, var deg, var year) ->
        System.out.printf("%s graduated %d with a %s\n", nm, year, deg);
      default -> {}
    }
  }
}
```

See Also

For a few nuances and for more on the rationale behind pattern matching and unpacking, see JEP 441.

8.10 Avoiding NPEs with “Optional”

Problem

You worry about null references causing a `NullPointerException` (NPE) in your code.

Solution

Use `java.util.Optional`.

Discussion

The computer scientist and professor who invented the notion of null pointers, a key early contributor to our discipline, has described the null reference as “my billion-dollar mistake”. However, use of `null` is not going away anytime soon.

What we can do is make clear that we worry about null pointers in certain contexts. For this purpose, Java 8 introduced the class `java.util.Optional`. The `Optional` is an object wrapper around a possibly null object reference. The notion of an `Optional` wrapper has a long history; a similar construct is found in several functional programming languages, including Scala and Haskell, and in LLVM’s ADT.

Java `Optional`s can be created with one of these methods:

`Optional.empty()`

: Returns an empty optional

`Optional.of(T obj)`

: Returns a nonempty optional containing the given value

`Optional.ofNullable(T obj)`

: Returns either an empty optional or one containing the given value

The basic operation of this class is to behave in one of two ways, depending on whether it is full or empty. Optional objects are immutable, so they cannot transition from one state to the other.

The simplest use is to invoke `isEmpty()` or its opposite, `isPresent()`, and use program logic to behave differently. This is not much different from using an `if` statement to check for null, but it puts the choice in front of you, making it less likely that you'll forget to check:

```
jshell> Optional<String> opt = Optional.of("What a day!");
opt ==> Optional[What a day!]

jshell> if (opt.isPresent()) {
   ...>     System.out.println("Value is " + opt.get());
   ...> } else {
   ...>     System.out.println("Value is not present.");
   ...> }
Value is What a day!
```

A better form would use the `orElse` method:

```
jshell> System.out.println("Value is " + opt.orElse("not present"));
Value is What a day!

jshell> opt = Optional.empty();
opt ==> Optional.empty

jshell> System.out.println("Value is " + opt.orElse("not present"));
Value is not present
```

One use case is that of passing values into methods. The object can be wrapped in an `Optional` either before it is passed to a method or after; the latter is useful when migrating from code that didn't use `Optional` from the start. The `Item` demo in Example 8-11 might represent part of a shipment-tracking program, a lending library manager, or anything else that has time-related data which might be missing.

Example 8-11. main/src/main/java/oo/OptionalDemo.java

```
    List.of(
      new Item("Item 1", LocalDate.now().plusDays(7)),
      new Item("Item 2")).
        forEach(System.out::println);
  static class Item {
    String name;
    Optional<LocalDate> dueDate;
```

```
    Item(String name) {
      this(name, null);
    }
    Item(String name, LocalDate dueDate) {
      this.name = name;
      this.dueDate = Optional.ofNullable(dueDate);
    }

    public String toString() {
      return "%s %s".formatted(name,
        dueDate.isPresent() ?
          "Item is due on " + dueDate.get() :
          "Sorry, do not know when item is due");
    }
  }
```

There are methods that throw exceptions, that return null, and so on. There are also methods for interacting with the `Streams` mechanism (see Recipe 9.3). These methods provide a major simplification, in that we don't have to check for null until we need to see the final result (for example, with Stream terminal methods returning `Optional`). A full list of `Optional`'s methods is at the start of the Javadoc page.

8.11 Controlling Subclassing with Sealed Types 17

Problem

You want to limit subclassing without making your types `final`.

Solution

Use sealed classes (or sealed interfaces).

Discussion

Until sealed classes were added, Java only allowed types to be `final` or to allow unlimited subclassing. With sealed classes, you can now provide an exhaustive list of subtypes; no other subtypes will be allowed.

Sealed types apply to classes and interfaces, but not to records, enums, or annotations.

The syntax is:

```
[public] sealed Type TypeName permits SubType1, SubType2, ...;
```

where the word `Type` can be either a class or an interface. Only the listed `SubType` classes will be allowed to directly extend `TypeName`. Subtypes of a sealed class must be declared either `sealed`, `non-sealed` (yes, that is a dash inside a Java language keyword), or `final`. Use:

- `final` if you want no further subclassing
- `sealed` with a `permits` list if you want controlled further subclassing
- `non-sealed` if you want uncontrolled subclassing from this particular type

A real benefit of this is that, when these types are used in a `switch` statement, the compiler can be sure that all possible types have been covered, obviating the need for a `default` case.

The code in Example 8-12 shows some examples.

Example 8-12. main/src/main/java/sealedclasses/simple_example.java

```
sealed class C1 permits C2, C3 {}  // Only 2 direct subclasses
final class C2 extends C1 {}    // cannot be subclassed
non-sealed class C3 extends C1 {}  // Valid subclass, wide open for subclassing!
class C4 extends C3 {}       // Subclass of C3
class SomeRandomUserClass extends C3 {} // Another subclass of C3
```

Figure 8-2 illustrates the possible subclasses from class `C1`.

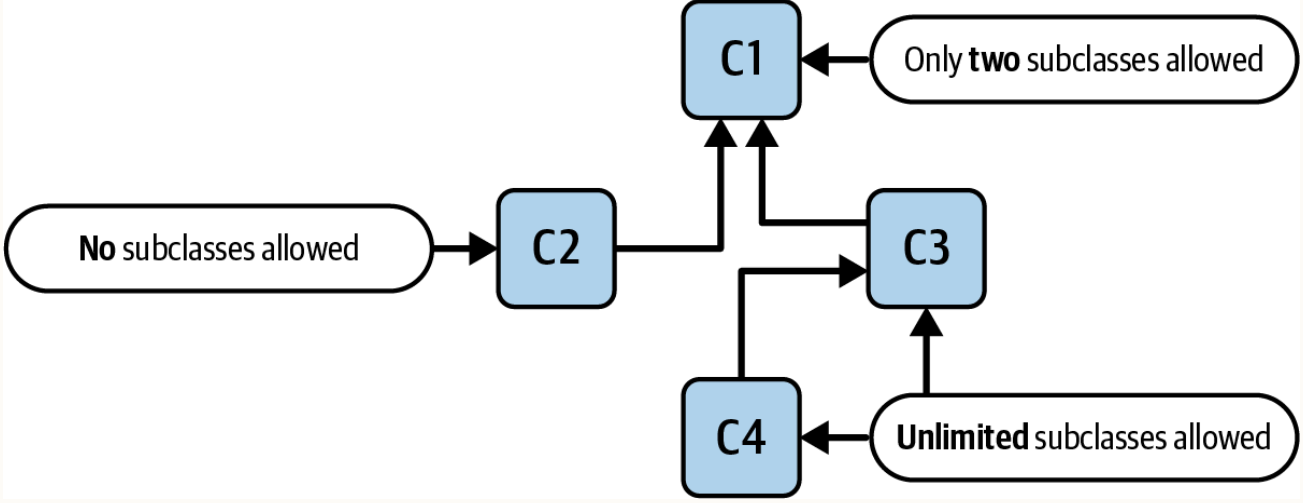


Figure 8-2. Sealed types inheritance

Sealed interfaces work the same way. A sealed interface only permits the named subinterfaces. The subinterfaces can be sealed or non-sealed, but not final, of course. This is illustrated by Example 8-13.

Example 8-13. main/src/main/java/sealedclasses/sealed_interfaces.java

```
void main() {
  System.out.println("Nothing to see here, folks.");
}

sealed interface A1 permits A2, A3, A4 {
  void process();
}

non-sealed interface A2 extends A1 {}

interface B1 extends A2 {}    // Allowed because A2 not sealed

non-sealed interface A3 extends A1 {}

sealed interface A4 extends A1 permits B2 {}

non-sealed interface B2 extends A4 {}
```

8.12 Enforcing the Singleton Pattern

Problem

You want to be sure there is only one instance of your class in a given Java Virtual Machine, or at least within your application.

Solution

There are several methods for making your class enforce the Singleton pattern:

- Using `enum` implementation
- Having only a private constructor (or multiple) and a `getInstance()` method
- Using a framework such as Spring or CDI (Recipe 8.14) configured to give Singleton-style instantiation of plain classes

Discussion

It is often useful to ensure that only one instance of a class gets created, usually to funnel all requests for some resource through a single point. An example of a Singleton from the standard API is `java.lang.Runtime`: you cannot create instances of `Runtime`; you simply ask for a reference by calling the static method `Runtime.getRuntime()`. Singleton is also an example of a design pattern that can be easily implemented. In all forms, the purpose of the Singleton implementation is to provide an instance in which certain methods can run, typically to control access to some resource.

The easiest implementation uses a Java `enum` to provide Singleton-ness. The `enum` mechanism already guarantees that only one instance of each enum constant will exist in a given JVM context, so this technique piggy-backs on that, as shown in Example 8-14.

Example 8-14. main/src/main/java/oo/EnumSingleton.java

```
public enum EnumSingleton {

    INSTANCE;

    // instance methods protected by singleton-ness would be here...

    /** A simple demo method */
    public String demoMethod() {
        return "demo";
    }
}
```

Using it is simple:

```
        // Demonstrate the enum method:
```

```
EnumSingleton.INSTANCE.demoMethod();
```

The next easiest implementation consists of a private constructor and a field to hold its result, as well as a static accessor method with a name like `getInstance()`.

The private field can be assigned from within a static initializer block or, more simply, by using an initializer. The `instance()` method (which should be public and must be static) then simply returns this instance, as shown in Example 8-15.

Example 8-15. main/src/main/java/oo/Singleton.java

```
public class Singleton {

    /**
     * Static initializer is run before class is available to code, avoiding
     * broken antipattern of lazy initialization in instance method.
     * For more complicated construction, could use static block initializer.
     */
    private static Singleton instance = new Singleton();

    /** A private Constructor prevents any other class from instantiating. */
    private Singleton() {
        // nothing to do this time
    }

    /** Static 'instance' method to obtain the one-and-only instance */
    public static Singleton instance() {
        return instance;
    }

    // other instance methods protected by singleton-ness would be here...

    /** A simple demo method */
    public String demoMethod() {
        return "demo";
    }
}
```

Note that the method of using *lazy evaluation* in the `getInstance()` method (as demonstrated in the book *Design Patterns*) is not necessary in Java because Java already uses *lazy loading*. Your `Singleton` class will probably not get loaded until its `getInstance()` is called, so there is no point in trying to defer the `Singleton` construction until it's needed

by having `getInstance()` test the `singleton` variable for null and creating the `Singleton` there.

Using this class is equally simple: just get the instance reference, and invoke methods on it:

```
// Demonstrate the codeBased method:
Singleton.instance().demoMethod();
```

Some commentators believe that a code-based Singleton should also provide a `public final clone()` method that just throws an exception, in order to avoid subclasses that cheat and `clone()` the Singleton. However, it is clear that a class with only a private constructor cannot be subclassed, so this paranoia does not appear to be necessary.

See Also

The `Collections` class in java.util has methods `singletonList()`, `singletonMap()`, and `singletonSet()`, which give out an immutable `List`, `Map`, or `Set`, respectively, containing only the one object that is passed to the method. This does not, of course, convert the object into a Singleton in the sense of preventing that object from being cloned or other instances from being constructed.

See discussions in the original *Design Patterns* book and the book *Effective Java*.

8.13 Roll Your Own Exceptions

Problem

You'd like to use an application-specific exception class or two.

Solution

Go ahead and subclass `Exception` or `RuntimeException`.

Discussion

In theory, you could subclass `Throwable` directly, but that's considered rude. You normally subclass `Exception` (if you want a checked exception) or `RuntimeException` (if

you want an unchecked exception). Checked exceptions are those that an application developer is required to either catch or delegate upward by listing them in the `throws` clause of the invoking method.

When subclassing either of these, it is customary to provide at least these constructors:

- A no-argument constructor
- A one-string argument constructor
- A two-argument constructor—a string message and a `Throwable` cause

The cause will appear if the code receiving the exception performs a stack trace operation on it, with the prefix `Root Cause is` or similar. Example 8-16 shows these three constructors for an application-defined exception, `ChessMoveException`.

Example 8-16. main/src/main/java/oo/ChessMoveException.java

```
/** A ChessMoveException is thrown  when the user makes an illegal move. */
public class ChessMoveException extends Exception {

  private static final long serialVersionUID = 8029117369881790 79L;

  public ChessMoveException () {
    super();
  }

  public ChessMoveException (String msg) {
    super(msg);
  }

  public ChessMoveException(String msg, Exception cause) {
    super(msg, cause);
  }
}
```

TIP

The Javadoc documentation for `Exception` lists a large number of subclasses; you might look there first to see if there is one you can use, before creating your own.

8.14 Using Dependency Injection

Problem

You want to avoid excessive coupling between classes, and you want to avoid excessive code dedicated to object creation/lookup.

Solution

Use a dependency injection (DI) framework.

Discussion

A DI framework allows you to have objects passed into your code instead of making you either create them explicitly (which ties your code to the implementing class name, since you're calling the constructor) or looking for them (which requires use of a possibly cumbersome lookup API, such as JNDI, the Java Naming and Directory Interface).

When using a DI framework, it's common to provide a Java interface for each class that you are working with, which this reduces coupling; that is, it removes dependencies on particular implementation classes and allows easier substitution of implementations when testing or when needs change or more flexibility is needed.

Three of the best-known DI frameworks are the Spring Framework (Spring DI), the Java Enterprise Edition's Contexts and Dependency Injection (CDI) (recently migrated to Jakarta), and Google Guice.

This example is a simplification of a numerical processing application. It uses Spring DI because it's relatively easy to work with. We have a main program that starts up Spring and requests an implementation of an interface called `Processor`. `Processor` does some numeric processing, which might be complex enough to use the Vector API described in Recipe 5.11. The `Processor` implementation is called `Process`, and because it is the top of the hierarchy, we'd have to either give its class name hardcoded in the main program, or provide an `@Configuration`-annotated class, which externalizes the dependency of the main program on the `Processor` implementation.

The `Processor` also needs a `Reporter` implementation to report its output; this might allow you to choose between a console reporter, a Swing reporter, a reporter that stores the results in a spreadsheet or database, etc. Given that the `Processor` has been ob-

tained from the Spring factory (`ApplicationContext`), we can use Spring DI's `@AutoWire` annotation to inject the `Reporter` implementation into it:

```
SpringMain - main program
`-- Processor - obtained from Context, via BeanConfig
   `-- Reporter - obtained automatically via @AutoWired
```

The `Processor` and `Reporter` interfaces are trivial; each has one method with a name similar to the interface name (see Example 8-17).

Example 8-17. Dependency interfaces

```
include: main/src/main/java/di0/Process.java
include: main/src/main/java/di0/Reporter.java
```

The main program is shown here:

main/src/main/java/di0/SpringMain.java

```
public class SpringMain {
 public static void main(String[] args) {
    ApplicationContext ctx =
      new AnnotationConfigApplicationContext("di0");

    Processor processor = ctx.getBean(Processor.class);

    processor.process(2,3);
 }
}
```

This class has to:

1. Set up the Spring context, which provides the DI framework.
2. Get the `Processor` from the context; it will already have the `Reporter` plugged into it by the DI framework.
3. Ask the `Processor` to process some data.

Furthermore, we don't depend on particular implementations of the interface.

How does Spring know to inject, or provide, a `Reporter` into the `Processor`? And what if there are multiple implementations of the `Reporter` interface? We have to tell it these things, which we'll do here with annotations:

```
@Configuration
public class SpringBeanConfig {
   @Bean
   public Processor processor() {
      return new Process();
   }
}
```

While Spring originally provided its own annotations, it will also accept the Java standard `@jakarta.annotation.Resource` annotation for injection and `@jakarta.inject.Named` to specify the injectee.

The `Processor` has the `Reporter` injected into it by the DI framework, just by asking for it with the `@AutoWired` annotation:

```
@Named("processor")
public class Process implements Processor {

      @Autowired
      private Reporter reporter;

      public void process(int x, int y) {
            var result = 2 * x + 3 * y;
            reporter.report(result);
      }
}
```

If you are familiar with Java design patterns, you might recognize this as a very simplified Model-View-Controller (MVC) application. A more explicit MVC version is in the *main/src/main/java/di* directory.

See Also

Spring DI, Java EE CDI, and Guice all provide powerful dependency injection. Spring's framework is more widely used; Java EE's framework has the same power and is built into every EE container. All three can be used alone or in a web application, with minor

variations. The overall Spring Framework provides special support for web apps, and CDI is already set up in Jakarta EE containers. There are many books on Spring. One book specifically treats Weld: *JBoss Weld CDI for Java Platform* by Ken Finnigan (O'Reilly).

8.15 Combining Java Features for Data-Oriented Programming

Problem

You want to make your applications more effective by combining such new features as `record`s, enhanced `switch` with pattern matching, and sealed types.

Solution

Learn the techniques of the data-oriented programming (DOP) paradigm.

Discussion

Like functional programming (Chapter 9), DOP is not a replacement for object-oriented programming, but an adjunct to it. However, DOP does walk back a bit from one key tenet of OOP: the combination of state and processing. Instead, DOP advocates using Java's `record` type as largely behaviorless data objects. Some of the main points are:

Modeling data as data

Use features like `record` (Recipe 8.4) and sealed classes (Recipe 8.11) for reliable data representation. This is intended to improve code readability and make it easier to reason about how data is used.

Separation of concerns

Treat data validation, data transport/storage, and data processing as distinct.

Immutable data

Make data objects immutable to increase reliability and make it easier to reason about data.

Pattern matching

Simplify data handling by using pattern matching in record unpacking and switches (with `instanceof`).

A simplified example is in Example 8-18. This provides a sample slice of an application used to bill customers for their orders, perhaps weekly.

Example 8-18. main/src/main/java/oo/DataOrientedDemo.java

```
sealed interface Transaction permits Order, Refund {} ❶

record Order(String product, int quantity,        ❷
 double price) implements Transaction {}

record Refund(String reason,
 double amount) implements Transaction {}

record Customer(String name, String address,      ❸
 String State, List<Transaction> orders) {}

void process(List<Customer> customers) {
 for (Customer c : customers) {
  double balance = 0;              ❹
  for (Transaction tx: c.orders) {
   switch (tx) {
   case Order(String p, int q, double price) -> { ❺
    System.out.printf("Invoice %s for %d %d for $%7.2f\n",
     c.name(), q, p, price);
    balance += price;
   }
   case Refund(String reason, double amount) -> {
    System.out.printf("Credit %s $%7.2f due to %s\n",
     c.name(), amount, reason);
    balance -= amount;
   }
   //                ❻
   }
  }
  System.out.printf("Net balance for %s is %f", c.name(), balance); ❼
 }
}

void main() {
 List<Transaction> txlist = List.of(new Order("Widgets", 10, 200.00),
  new Order("Whatzits", 5, 125.00),       ❽
  new Refund("Lost shipment #456", 175.00));
```

```
    var cust =
      new Customer("Whizzy Systems Inc", "123 Erewhon St", "Confusion", txlist);
    process(List.of(cust));
  }
```

❶ Using a `seal`ed interface lets the compiler be sure we handle all cases.

❷ Records can implement interfaces; we implement `Transaction` here.

❸ `Customer` is also a record, making all our data types immutable and thus thread-safe.

❹ Set up for item 5.

❺ Processing is done here, not in methods of `Order`, `Refund`, or `Customer`.

❻ No `default` case; see item 1.

❼ Printing out the balance concludes the processing started in item 5.

❽ This dummy data stands in for code that would receive *and validate* the `Transaction` and `Customer` objects, making our data "safe" in the rest of the code.

See Also

Brian Goetz's InfoQ article on DOP was part of the Java team's push to promote the use of DOP. Another Java team member, Nicolai Parlog, wrote this six-part series on DOP.

1 See Alexander et al., *A Pattern Language*, Oxford University Press, 1977.

2 A value class is one used mainly to hold state, rather than logic: a `Person` is a value class, whereas `java.lang.Math` is not. Many classes are somewhere in between.

3 Andrew Slice points out that this reminds him of a common quote: "It's going to be the next guy's problem, and the next guy is probably you."

4 Of course these lights wouldn't have remote power-off. But the computers might, for maintenance purposes.

[5] For Java folks not that familiar with C/C++, C is the older, non-OO language; C++ is an OO derivative of C; and Java is in part a portable, more strongly typesafe derivative of C++.

Previous chapter
< 7. Structuring Data with Java

Next chapter
9. Functional Programming Techniques: Functional Interfaces, Streams, and Parallel Collections >